

KNOWLEDGE REPRESENTATION

Lecture 5: Reasoning about Concurrent Actions

dr Anna Maria Radzikowska

Warsaw University of Technology
Faculty of Mathematics and Information Science
Building MiNI PW, room 504

E-mail: Anna.Radzikowska@pw.edu.pl

Warsaw 2026



Outline

- ➊ Problems: conflicts and actions' executability
- ➋ Effects of concurrent actions: inheritance and restricted inheritance
- ➌ Action Language $\mathcal{AC}$ – Syntax
- ➍ Conflicting actions
- ➎ Executability of actions
- ➏ Decomposition of actions
- ➐ Resulting states
- ➑ Semantics of $\mathcal{AC}$
 - Structures
 - Models
 - Example: Producer & Consumers

Concurrent actions

Conflict

What does it mean *conflicting actions*? In the theory of concurrency it is assumed that two processes accessing the same source are conflicting. In general case, however, this approach seems too restrictive. For example, two robots can pick up a heavy block simultaneously. Moreover, this action may be unexecutable by each robot separately.

Executability of actions

The conflict problem is a particular case of the one of actions' executability – clearly, conflicting actions cannot be executed concurrently. However, actions cannot be executed due to other reasons. For example, if there is a distance between the window and the door, I am not able to open them simultaneously.

Two examples: effects of concurrent actions

Example 5.1

Mary is lifting a bowl of soup from the kitchen table, while John is opening the door to the dining room.

These two actions are independent, so they can be performed simultaneously leading to the effect being the conjunction of effects of particular actions (i.e., the door is open and the bowl is lifted) – this way the *compound* action inherits results from its subactions.

Example 5.2

Whenever Mary tries to lift the bowl with one hand, she spills the soup (but lifted the bowl). But when she uses both hands, she does not spill the soup. We know that the soup is not spilled initially and the bowl is not lifted. Now the effect “*the soup is spilled*” is not inherited when two actions, `LIFTLEFT` and `LIFTRIGHT`, are executed simultaneously.

Effects of concurrent actions

Let D be an action domain and let the compound action $A = \{A_1, \dots, A_k\}$, $k \geq 2$, be given.

Inheritance

If there is no statement in D describing effects of any subset of A , the concurrently executed actions lead to the effect being the conjunction of effects of particular actions.

Restricted inheritance

If there is an action description statement in D referring to some subset $A' \subseteq A$, then the inheritance rule is not applied for elements of A' : A' leads to the effect given in statements in D referring to A' and no effect of the actions A' is composed of is taken into account. In other words, if such a statement is mentioned in D , it constitutes a “new action” (wrt its results).

Syntax

- **Signature**: a pair $\Upsilon = (\mathcal{F}, \mathcal{Ac})$ where $\mathcal{F}$ is the set of **fluents** and $\mathcal{Ac}$ is the set of **atomic actions** (atoms).
- A **compound action** is any set $A \subseteq \mathcal{Ac}$ of atoms.
- A **statement** of $\mathcal{AC}$ is any statement of $\mathcal{AR}$ where instead of atomic actions we use compound actions, referred to as **actions**.
- An **action domain** is a non-void set of statements.

Elementary actions

Let D be an action domain. an **elementary action** is any action $A \subseteq \neg c$ such that D contains any of the following statements:

A **causes** α **if** π

A **releases** f **if** π .

The set of all elementary actions in D will be denoted by $Elem(D)$.

Conflicting actions

Conflicting actions

Example 5.3

Consider the following two statements:

$$\begin{aligned} A & \textbf{causes} f \vee g; \\ B & \textbf{causes} f \end{aligned}$$

where A and B are atomic actions.

Let $\sigma = \{\neg f, \neg g\}$. Performing A in σ leads to two states:

$$\sigma_1 = \{p, \neg g\}, \quad \sigma_2 = \{\neg f, g\}$$

while performing B in σ leads to σ_1 .

If we admit parallel execution of A and B in σ , then the resulting state will be σ_1 .

Thus composing a nondeterministic action with a deterministic one we obtain a deterministic action!

Conflicting actions (cont.)

Example 5.4

Consider the following statements:

A **causes** $f \vee g$;

B **releases** f .

Again, let $\sigma = \{\neg f, \neg g\}$. Performing A leads to either of these states:

$$\sigma_1 = \{f, \neg g\}, \quad \sigma_2 = \{\neg f, g\},$$

while executing B in σ leads to either σ or σ_1 .

Again, if we admit performing $\{A, B\}$, the resulting state will be σ_1 .

Then the composition of two deterministic actions makes a deterministic action!

Conflicts – problematic cases

Case 1

D contains the following statements:

A **causes** α **if** π

B **releases** f **if** ψ

where

- $\alpha \Leftrightarrow \gamma_1 \vee \dots \vee \gamma_k$;
- $\gamma_i = \lambda_i^1 \wedge \dots \wedge \lambda_i^{k_i}$, where $\lambda_i^{k_i}$ is a literal;
- there is some $i = 1, \dots, k$ and $j = 1, \dots, k_i$ such that $f = \overline{\lambda_i^j}$.

In other words, f occurs in at least one possible effect of performing A in a state satisfying $\pi \wedge \psi$.

Conflicts – problematic cases

Case 2

D contains the following statements:

A **causes** α **if** π

B **causes** β **if** ψ

where

- $\alpha \Leftrightarrow \gamma_1 \vee \dots \vee \gamma_k$ and $\beta \Leftrightarrow \delta_1 \vee \dots \vee \delta_m$;
- each $\gamma_i = \lambda_i^1 \wedge \dots \wedge \lambda_i^{k_i}$ and $\delta_p = \tau_p^1 \wedge \dots \wedge \tau_p^{k_p}$ where λ_i^j and τ_p^l are literals;
- there is some $i = 1, \dots, k$ and $p = 1, \dots, m$ such that γ_i and δ_p contain complementary literals (i.e., f and $\neg f$).

In other words, there is a possible effect of A (γ_i) **contradictory** to some possible effect of B (δ_p), both actions performed a state satisfying $\sigma \models \pi \wedge \psi$.

Disjunctive Normal Form

Let α be any formula over $\mathcal{F}$. We say that α **is in DNF** iff

$$\alpha = \bigvee_{i=1}^n \bigwedge_{j=1}^{k(i)} \lambda_{ij},$$

where λ_{ij} is a literal, $i = 1, \dots, n$, $j = 1, \dots, k(i)$.

E.g., $\alpha_1 = (p \wedge q) \vee \neg r$ is in DNF.

Theorem 5.1

Every formula α can be uniquely transformed to the formula α' in DNF such that $\alpha \Leftrightarrow \alpha'$.

Notation: The formula resulting from transformation of α into DNF will be denoted **DNF(α)**.

For $DNF(\alpha) = \gamma_1 \vee \dots \vee \gamma_k$, each γ_i is called a **component of** α .

Converting to DNF

Algorithm

- 1 Eliminate $\rightarrow$ and $\Leftrightarrow$:

$$\alpha \rightarrow \beta \rightsquigarrow \neg\alpha \vee \beta$$

$$\alpha \Leftrightarrow \beta \rightsquigarrow (\neg\alpha \vee \beta) \wedge (\neg\beta \vee \alpha).$$

- 2 Move $\neg$ to the right:

$$\neg(\alpha \wedge \beta) \rightsquigarrow \neg\alpha \vee \neg\beta$$

$$\neg(\alpha \vee \beta) \rightsquigarrow \neg\alpha \wedge \neg\beta$$

$$\neg\neg\alpha \rightsquigarrow \alpha.$$

- 3 Distribution $\vee$ over $\wedge$:

$$\alpha \wedge (\beta \vee \gamma) \rightsquigarrow (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$$

$$(\beta \vee \gamma) \wedge \alpha \rightsquigarrow (\beta \wedge \alpha) \vee (\gamma \wedge \alpha).$$

Conflicting actions

Let D be an action domain, let σ be a state, and let $A, B \subseteq \mathcal{A}_c$ be two actions. We say that A and B are **conflicting in σ** iff at least one of the following holds:

Case 1

There are two statements in D of the form

A **causes** α **if** π ;

B **causes** β **if** ψ

such that

- $\sigma \models \pi \wedge \psi$;
- some component of α is inconsistent with some component of β .

Case 2

There are two statements in D of the form:

A **causes** α **if** π ;
 B **releases** f **if** ψ

such that

- $\sigma \models \pi \wedge \psi$;
- f occurs in some component of α .

Executability of actions

Executability of actions

Lack of conflict does not mean that actions are executable in a state.

Example 5.5

Consider the following action domain:

PAINTDOOR **causes** $pDoor$;
PAINTWALL **causes** $pWall$;
PAINTWINDOW **causes** $pWindow$;
impossible {PAINTDOOR, PAINTWINDOW}.

Let $\sigma = \{\neg pDoor, \neg pWindow, \neg pDoor\}$. Is it possible to execute $A = \{\text{PAINTWALL}, \text{PAINTWINDOW}, \text{PAINTDOOR}\}$ in σ ?

Executability of actions (cont.)

Two possibilities

- 1 Neither A nor any of its supset is executable;
- 2 The action A becomes a non-deterministic: either $\{\text{PAINTDOOR}, \text{PAINTWALL}\}$ or $\{\text{PAINTWINDOW}, \text{PAINTWALL}\}$ is actually performed.

In the following we will adopt the second case!

Executability of actions

Potentially executable actions

Let D be an action domain and let σ be a state. An action $A = \{A_1, \dots, A_k\}$ with $A_i \in \text{Elem}(D)$ is **potentially executable in σ** iff there is no an action $B \subseteq \bigcup_{i=1}^k A_i$ such that D contains statements

B **causes** α_1 **if** π_1 ;

...

B **causes** α_n **if** π_n

such that

- $\sigma \models \pi_1 \wedge \dots \wedge \pi_n$;
- $\alpha_1 \wedge \dots \wedge \alpha_n$ is inconsistent.

Potentially executable actions

Example 5.6

Consider the following action domain:

A **causes** p **if** q ;
 B **causes** $\neg q$ **if** q ;
 C **causes** s **if** q ;
 $\{A,B\}$ **causes** $\neg r$ **if** q ;
impossible $\{A,C\}$ **if** r .

Let $\sigma \models q \wedge r$. Note that elementary actions $\{A,B\}$ and $\{C\}$ are not potentially executable since $\{A,B\} \cup \{A,C\}$ leads to $\neg r \wedge \perp$.

However, $\{B\}$ and $\{C\}$ are potentially executable.

Decomposition of actions

Decomposition of actions

Let D be an action domain, let σ be a state, and let $A \subseteq \mathcal{Ac}$. A **decomposition of A in σ** is a set $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ of elementary actions in D such that

(D1) $\bigcup_{i=1}^k E_i \subseteq A$;

(D2) $\Delta(A, \sigma)$ is the set of non-conflicting actions potentially executable in σ ;

(D3) there is no set $\{E'_1, \dots, E_k, \dots, E_m\}$ of elementary actions satisfying the above two conditions and, in addition, $E_i \subset E'_i$ for some $i = 1, \dots, k$

Note that **(D3)** states the maximality of $\Delta(A, \sigma)$.

Components

If $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ is a decomposition of A in σ , then each E_i , $i = 1, \dots, k$, is a **component of the decomposition Δ of A in σ** .

Example 5.7

Consider the following action domain D :

A **causes** $\neg p$ **if** p ;
B **causes** q **if** p ;
C **causes** r **if** p ;
D **causes** w **if** p ;
 $\{A,B\}$ **causes** v **if** p ;
 $\{B,C\}$ **causes** s **if** p ;
 $\{A,B,C\}$ **causes** f **if** p ;
impossible $\{A,C\}$.

Let $E = \{A,B,C,D\}$ and let $\sigma \models p$. There are two decompositions of E in σ , namely

$$\Delta_1(E, \sigma) = \{\{A,B\}, \{D\}\}$$

$$\Delta_2(E, \sigma) = \{\{B,C\}, \{D\}\}.$$

Resulting states

Potentially resulting states

Let D be an action domain, let σ, σ' be two states, let $A \subseteq \mathcal{A}_c$, and let $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ be a decomposition of A in σ . We say that σ' is a **potentially resulting state of executing A in σ** iff for every $i = 1, \dots, k$ and for every statement

$$(E_i \text{ *causes* } \alpha \text{ *if* } \pi) \in D,$$

$\sigma \models \pi$ implies $\sigma' \models \alpha$.

Intuitively, while performing A in σ , we get a (potentially) resulting state satisfying all effects of its components E_i in σ which belong to the same decomposition.

Resulting states (cont.)

Potentially resulting states

In other words, if A has n distinct decomposition:

$$\Delta_1(A, \sigma) = \{E_1^1, \dots, E_{k_1}^1\}$$

...

$$\Delta_n(A, \sigma) = \{E_1^n, \dots, E_{k_n}^n\}$$

and the execution of E_i^j in σ has the effect α_i^j , then the potentially resulting state σ' (of performance of A in σ) satisfies

$$(\alpha_1^1 \wedge \dots \wedge \alpha_{k_1}^1) \vee \dots \vee (\alpha_1^n \wedge \dots \wedge \alpha_{k_n}^n).$$

Resulting states (cont.)

Example 5.7 (cont.)

A *causes* $\neg p$ *if* p ;
B *causes* q *if* p ;
C *causes* r *if* p ;
D *causes* w *if* p ;
 $\{A,B\}$ *causes* v *if* p ;
 $\{B,C\}$ *causes* s *if* p ;
 $\{A,B,C\}$ *causes* f *if* p ;
impossible $\{A,B\}$.

Executing $E=\{A,B,C,D\}$ in σ potentially leads to two states σ' and σ'' such that

$$\sigma' \models v \wedge w$$

$$\sigma'' \models s \wedge w.$$

Hence, after E , we have for sure w and either v or s .

Structures

Structure

Let $\mathcal{L}$ be a language of the class $\mathcal{AC}$. A **structure for $\mathcal{L}$** is a triple (Σ, σ_0, Res) where Σ is a set of states, $\sigma_0 \in \Sigma$ is the initial state, and $Res : 2^{\mathcal{AC}} \times \Sigma \rightarrow 2^\Sigma$ is a transition function.

Function Res_0

Let D be an action domain in $\mathcal{L}$ of the class $\mathcal{AC}$. Define an auxiliary function $Res_0 : 2^{\mathcal{AC}} \times \Sigma \rightarrow 2^\Sigma$ for every $A \subseteq \mathcal{AC}$ and for every $\sigma \in \Sigma$, if A in σ has n decompositions $\Delta_1(A, \sigma), \dots, \Delta_n(A, \sigma)$ with $\Delta_i(A, \sigma) = \{A_1^i, \dots, A_{k_i}^i\}$, then $Res_0(A, \sigma)$ is the set of all potentially resulting states of performing A in σ .

Function New

for any $A \subseteq \mathcal{AC}$, for any $\sigma \in \Sigma$, and for any $\sigma' \in Res_0(A, \sigma)$, $New(A, \sigma, \sigma')$ is defined as in the semantics of $\mathcal{AR}$.

Model

Let D be an action domain in a language $\mathcal{L}$ of the class $\mathcal{AC}$. A structure $S = (\Sigma, \sigma_0, Res)$ is **model** of D iff

- (M1) Σ is the set of all states of D , i.e., the set of all states satisfying all constraints in D ;
- (M2) each value statement and each observation statement in D is true in S (defined analogously as in $\mathcal{AR}$);
- (M3) for every $A \subseteq \mathcal{AC}$ and for every $\sigma \in \Sigma$, $Res(A, \sigma) \subseteq Res_0(A, \sigma)$ is the set of all states $\sigma' \in \Sigma$ for which the sets $New(A, \sigma, \sigma')$ are minimal wrt set inclusion.

Example: Producer & Consumers

Example 5.8

There is a producer and two consumers, A and B. The producer puts to the buffer as many items of his product as possible by performing an action PUT. Each consumer can perform two actions CONSUME and GET. The action GET makes the buffer empty and the consumer has a product, while the action CONSUME causes that the consumer does not have an item.

REPRESENTATION IN $\mathcal{AC}$:

PUT **causes** $\neg \text{empty}$;
GETA **causes** $\text{empty} \wedge \text{hasA}$ **if** $\neg \text{empty}$;
GETB **causes** $\text{empty} \wedge \text{hasB}$ **if** $\neg \text{empty}$;
CONSUMEA **causes** $\neg \text{hasA}$ **if** hasA ;
CONSUMEB **causes** $\neg \text{hasB}$ **if** hasB ;
impossible {GETA, GETB}.

Example: Producer & Consumers (cont.)

Example 5.8 (cont.)

- Consider $\sigma = \{hasA, hasB, \neg empty\}$ and the action $A = \{PUT, GETA, GETB, CONSUMEA, CONSUMEB\}$.
- The actions PUT and GETA, as well as the actions PUT and GETB, are conflicting in σ , thus executing PUT makes GETA and GETB unexecutable.
- Similarly, GETA and GETB are together unexecutable and either of these action can be performed at a time.
- Moreover, GETA and CONSUMEA, as well as GETB and CONSUMEB, are conflicting.

Example: Producer & Consumers (cont.)

Example 5.8 (cont.)

Hence, we obtain the following three decompositions of A in σ :

$$\Delta_1(A, \sigma) = \{\text{PUT}, \text{CONSUMEA}, \text{CONSUMEB}\}$$

$$\Delta_2(A, \sigma) = \{\text{GETA}, \text{CONSUMEB}\}$$

$$\Delta_3(A, \sigma) = \{\text{GETB}, \text{CONSUMEA}\}$$

Therefore, $\text{Res}(A, \sigma) = \{\sigma_1, \sigma_2, \sigma_3\}$ where

$$\sigma_1 = \{\neg \text{hasA}, \neg \text{hasB}, \neg \text{empty}\}$$

$$\sigma_2 = \{\text{hasA}, \neg \text{hasB}, \text{empty}\}$$

$$\sigma_3 = \{\neg \text{hasA}, \text{hasB}, \text{empty}\}.$$

Thank you for your attention!

Questions are welcome.